

Graph Neural Networks in Molecular Insights

March 3, 2025

Maibam Rakesh Singh

Maibiya R. KhumanCha

1 Introduction

Graph-structured data is fundamental to many scientific and real-world domains, including molecular chemistry, biology, social networks, and knowledge graphs. Unlike traditional tabular data, graphs capture both node attributes and the relationships that connect them, enabling richer and more expressive modeling. Graph Neural Networks (GNNs) have emerged as a powerful family of models capable of leveraging this structure through message passing, where information flows along edges and node representations are updated based on their neighbors. This process allows GNNs to incorporate both local and multi-hop interactions, making them especially effective for tasks such as molecular property prediction, where chemical context and structural dependencies play a critical role.

Despite their effectiveness, GNNs often behave as black-box models. Their predictions may rely on intricate combinations of substructures, edge patterns, and feature interactions that are not immediately interpretable. In chemistry-related applications, interpretability is essential for validating whether a model has learned chemically meaningful motifs, such as aromatic rings, functional groups, or mutagenic substructures, rather than exploiting dataset-specific quirks. Understanding why a GNN predicts a molecule to be mutagenic is crucial not only for scientific insight but also for ensuring reliability and trustworthiness in downstream decision-making.

To provide such transparency, this lab focuses on GNNExplainer, a general-purpose framework for interpreting GNN predictions. Instead of analyzing an entire molecular graph, GNNExplainer identifies a compact subgraph and a minimal subset of node features that most strongly influence the model's output. In practice, this means highlighting the specific atoms and bonds that drive a prediction, revealing the underlying chemical reasoning embedded within the learned model. By isolating these influential components, GNNExplainer makes it possible to connect model behavior with domain knowledge in a clear and intuitive way.

In this lab, you will work with the MUTAG dataset, a benchmark collection of molecular graphs labeled by their mutagenicity. You will train a Graph Convolutional Network (GCN) for graph-level classification and then apply GNNExplainer to uncover which molecular substructures contribute to its decisions. Throughout the process, you will visualize important edges, nodes, and faithful subgraphs, and you will analyze how explanation fidelity changes as the graph is sparsified. These insights will help you understand how the model processes chemical information and how explainability techniques enhance transparency in graph learning.

1.1 Objectives

After completing this lab, you will be able to:

- Explain how graph-structured molecular data can be represented using nodes (atoms) and edges (chemical bonds).
- Train a Graph Convolutional Network (GCN) in PyTorch Geometric for graph-level classification on the MUTAG dataset.
- Apply GNNExplainer to identify the most influential substructures that drive a GNN’s prediction.
- Visualize important atoms, bonds, and faithful subgraphs that contribute to mutagenicity classification.
- Evaluate explanation quality using fidelity curves and sparsity-based interpretation metrics.

1.2 Preliminaries

1.2.1 Graph Neural Networks (GNNs)

Graph Neural Networks (GNNs) are designed to learn from graph-structured data, where nodes represent entities (e.g., atoms or users) and edges represent relationships (e.g., chemical bonds or user-item interactions). Through a process known as message passing, each node iteratively updates its representation by aggregating information from its neighbors.

Figure 1 illustrates the concept of multi-hop neighborhood aggregation in GNNs. The target node u aggregates feature representations from its 1-hop, 2-hop, and 3-hop neighbors, enabling information propagation across multiple layers and capturing higher-order structural dependencies within the graph.

At layer $k + 1$, the representation of node u is updated as:

$$h_u^{(k+1)} = \text{AGG}(h_u^{(k)}, \{h_v^{(k)} : v \in \mathcal{N}(u)\}),$$

where $h_u^{(k)}$ denotes the embedding of node u at layer k , $\mathcal{N}(u)$ is the set of neighbors of u , and $h_v^{(k)}$ represents the embeddings of neighboring nodes.

The function $\text{AGG}(\cdot)$ is a permutation-invariant aggregation operator (e.g., mean, sum, or max), ensuring that the result does not depend on the ordering of neighbors. This formulation captures the core principle of GNNs: node representations are learned by aggregating information from their local neighborhoods, enabling the model to encode both structural and relational context within the graph.

1.2.2 Graph Convolutional Networks (GCNs)

A Graph Convolutional Network (GCN) propagates information through the graph using a symmetrically normalized adjacency matrix. The layer-wise update rule is given by:

$$H^{(k+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(k)}W^{(k)}\right)$$

where $\tilde{A} = A + I$ denotes the adjacency matrix with added self-loops, and $\tilde{D}$ is the corresponding degree matrix.

This formulation can be interpreted as a sequence of operations that define a graph convolution. The addition of self-loops ensures that each node incorporates its own features during message passing. The symmetric normalization term $\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}$ rescales messages based on node degrees, preventing high-degree nodes from dominating the aggregation and promoting stable training across layers. The matrix $H^{(k)}$ represents the node embeddings at layer k , which are linearly transformed by the trainable weight matrix $W^{(k)}$. Finally, the nonlinear activation function $\sigma(\cdot)$ introduces additional expressive power, allowing the model to learn complex patterns over the graph.

Overall, this equation formalizes how node representations are updated by aggregating and transforming information from neighboring nodes in a mathematically normalized and stable way, which is the defining characteristic of Graph Convolutional Networks.

While GCNs were originally developed for generic graph learning tasks, their message-passing framework naturally extends to molecular graphs, where atoms and bonds form structured relational data.

1.2.3 Molecular Graphs

Molecules can be represented as graphs in which nodes correspond to atoms (with properties such as atomic number, aromaticity, and valence) and edges correspond to chemical bonds (single, double, or aromatic). This structure enables GNNs to capture how electronic and functional relationships propagate across the molecule.

Figure 2 shows a simple molecular graph illustrating how chemical structures map naturally to graph representations, with atoms as nodes and bonds as edges.

In the MUTAG dataset, several functional groups play key roles in mutagenicity classification, including NO (nitro) groups, NH (amino) groups, and aromatic rings, all of which strongly influence electron distribution and chemical reactivity.

1.2.4 GNNExplainer

Overview: GNNExplainer provides local explanations by identifying an explanation subgraph G_S and important node features X_S^F . This highlights the atoms, bonds, and feature dimensions most responsible for a model’s prediction.

Figure 3 presents an overview of the GNNExplainer framework (adapted from Ying et al., 2019). The explainer learns continuous masks over edges and node features, revealing which message-passing pathways and atomic attributes most strongly influence the GNN’s prediction. The resulting explanation subgraph highlights the functional groups and structural regions responsible for the model’s decision.

Optimization Objective: At a high level, GNNExplainer answers the question: What is the smallest subgraph that still leads the model to make the same prediction? This objective is formalized by maximizing the mutual information between the model’s prediction Y and a compact explanation subgraph G_S :

$$\max_{G_S} I(Y; G_S) = H(Y) - H(Y | G_S),$$

where $H(Y)$ denotes the uncertainty of the model’s prediction, and $H(Y | G_S)$ is the uncertainty after conditioning on the explanation subgraph.

In practice, $H(Y)$ is constant for a trained model, so maximizing mutual information reduces to minimizing the conditional uncertainty $H(Y | G_S)$. If G_S explains the prediction well, the conditional uncertainty is small. Therefore, a good explanation subgraph is one that preserves the model’s confidence while remaining compact.

Continuous Masking: To make the explanation objective differentiable, GNNExplainer introduces a continuous edge mask:

$$M \in [0, 1]^{|E|},$$

where each value M_e represents the importance assigned to edge e . Applying this mask to the original adjacency matrix yields a masked graph:

$$A_M = A \odot M,$$

with $\odot$ denoting element-wise multiplication. This continuous relaxation allows the explainer to learn edge importance via gradient-based optimization, enabling end-to-end training of the explanation masks.

Training Loss: GNNE explainer minimizes the following objective:

$$\mathcal{L} = -\log P_{\theta}(Y \mid G \odot M) + \lambda_1 \|M\|_1 + \lambda_2 H(M),$$

which balances prediction fidelity with explanation simplicity.

The loss consists of three components. The fidelity term $-\log P_{\theta}(Y \mid G \odot M)$ encourages the masked graph to preserve the model’s original prediction, ensuring that the learned explanation remains faithful to the decision boundary of the underlying GNN. The sparsity term $\lambda_1 \|M\|_1$ promotes solutions in which only a small subset of edges receives non-zero importance, driving the explainer toward parsimonious subgraphs that capture only the most influential structural components. The entropy regularization term $\lambda_2 H(M)$ penalizes high-entropy masks, discouraging diffuse or ambiguous importance assignments and pushing mask values toward binary-like decisions that yield crisp and interpretable explanation subgraphs.

After optimization, thresholding $\sigma(M)$ produces the final explanation subgraph G_S .

Edge and Feature Masks: GNNE explainer learns two complementary types of explanation masks: an edge mask, which identifies the most influential edges (e.g., chemical bonds), and a feature mask, which highlights the most important node feature dimensions (e.g., atomic properties).

For molecular prediction tasks, the edge mask typically provides the clearest and most intuitive interpretation, as it directly reveals chemical motifs and functional groups that drive the model’s decision.

Minimal Faithful Subgraphs: An explanation is considered faithful when the model produces nearly the same prediction using only the explanation subgraph:

$$f(G_S) \approx f(G),$$

where $f(\cdot)$ denotes the trained GNN’s prediction function.

Intuitively, this means that the GNN arrives at almost the same decision even when restricted to the explanatory subgraph. By progressively removing edges with low importance scores, one can generate fidelity curves that track how prediction confidence changes as the graph is pruned. The smallest subgraph that preserves the original prediction is referred to as the minimal faithful subgraph.

In the MUTAG dataset, such minimal faithful subgraphs often correspond to well-known mutagenic motifs, such as NO (nitro) groups attached to aromatic rings, providing chemically meaningful explanations for the model’s predictions.

2 Experiments

2.1 Installing Libraries

In this first step, you will install all required libraries for graph learning and explainability. We install PyTorch, PyTorch Geometric (PyG), and supporting utilities for visualization and data processing. PyG is essential for loading the MUTAG dataset, defining GCN layers, and running GNNExplainer.

After installation, you may need to restart the kernel to ensure all packages load correctly.

2.2 Imports and Reproducibility

Here we import all necessary Python libraries for the project, including PyTorch, PyG, NumPy, NetworkX, and plotting utilities. We also configure deterministic behavior by setting seeds for reproducibility.

2.3 Loading the MUTAG Dataset

In this step, you download and prepare the MUTAG dataset for graph classification. MUTAG is a benchmark dataset of nitroaromatic molecules labeled as mutagenic or non-mutagenic. PyG automatically processes raw files into a list of molecular graphs. We display dataset statistics, including node feature dimensions and class distribution.

The dataset consists of 188 molecular graphs with 7 node features and 2 classes. The label distribution shows 125 mutagenic compounds (class 1) and 63 non-mutagenic compounds (class 0).

2.4 Train-Validation Split and Data Loaders

We create training and validation splits using stratified sampling to preserve class balance. Each molecule is treated as one graph-based example. PyG's DataLoader batches graphs into mini-batches for efficient training. This builds the input pipeline that the GCN will use during training.

The stratified 80/20 split results in 150 training graphs and 38 validation graphs, maintaining the class distribution with 50 mutagenic and 100 non-mutagenic samples in the training set, and 13 mutagenic and 25 non-mutagenic samples in the validation set.

2.5 Defining the Graph Convolutional Network (GCN)

This defines the GCN architecture used for MUTAG graph-level prediction. The model consists of two graph convolution layers with batch normalization, ReLU activation, and dropout for regularization. A global mean pooling layer aggregates node embeddings into a graph-level representation. The final linear layer outputs logits for binary classification.

This GCN will later be explained using GNNExplainer. The architecture takes input features of dimension 7, processes them through two hidden layers with 96 neurons each, and outputs 2 classes for binary classification.

2.6 Training the GCN

Here we train the model using cross-entropy loss and the Adam optimizer. A cosine annealing scheduler helps stabilize learning. We apply early stopping based on validation accuracy to avoid overfitting on the small MUTAG dataset. After training finishes, the best set of model weights is restored.

The training runs for up to 100 epochs with a patience of 10 for early stopping. The best validation accuracy of 0.763 is achieved at epoch 13, after which training stops early to prevent overfitting.

2.7 Running GNNExplainer on a Validation Graph

In this step, you select one molecular graph from the validation set and generate a local explanation. The explainer returns importance scores for edges and node features, and the model's predicted class. We also compute the minimal faithful subgraph, the smallest edge set that maintains the original prediction.

This step demonstrates the core interpretability workflow. For a selected validation graph, the base prediction is class 0, and the minimal faithful subgraph requires only 25.0% of the original edges to preserve this prediction.

2.8 Visualizing Explanation Outputs

This cell generates visualizations showing:

- The entire graph with edge importance values
- The top 25% most important edges
- The minimal faithful subgraph

These plots reveal how the GNN reasons about molecular structure by highlighting compact atomic neighborhoods that strongly influence the prediction. Because the simplified MUTAG representation does not include bond order, aromaticity, or explicit functional-group descriptors, the explanations focus on local patterns such as nitrogen (N) or oxygen (O) atoms with high-importance edges and carbon-rich cycles where many C atoms form ring-like patterns, rather than fully reconstructed motifs like NO or NH. The underlying MUTAG dataset still consists of nitroaromatic compounds labeled as mutagenic or non-mutagenic, and these visualizations help assess whether the model attends to chemically plausible regions of each molecule.

2.9 Displaying Multiple Explanations

Here we run GNNExplainer on several validation graphs and display them in a grid. This allows you to compare explanations across molecules and identify recurring mutagenic patterns. Up to 6 validation graphs are displayed in a 3-column grid format.

2.10 Computing the Fidelity Curve

This computes the fidelity curve for the selected graph by progressively retaining only the top- k most important edges, as identified by the explainer, and tracking whether the model’s original prediction is preserved. The curve illustrates how prediction stability changes as structural information is removed or reintroduced.

Because prediction fidelity depends on the presence of specific critical edges or structural motifs, rather than simply the number of retained edges, the curve can exhibit non-monotonic behavior with multiple drops and rises. Removing a key edge or disrupting a supporting message-passing path may cause the prediction to change, resulting in a drop in fidelity. As k increases, reintroducing that edge or an alternative sufficient substructure can restore the original prediction, leading to a rise in fidelity.

The minimal faithful ratio corresponds to the smallest value of k at which the prediction is preserved, identifying the smallest subgraph that is sufficient to explain the model’s decision. For the selected graph, the minimal faithful ratio is 0.250.

2.11 Computing Explanation Statistics Across All Validation Graphs

In this final analysis step, we run GNNExplainer on every validation graph and compute the minimal faithful subgraph size for each. We track predicted class, number of edges, and minimal edge set required for explanation. This provides insight into how complex or compact explanations are across the validation set.

The analysis reveals that minimal faithful subgraph ratios vary across validation graphs, ranging from approximately 0.182 to 0.375, with total edges ranging from 14 to 24 undirected edges. This variation reflects the differing structural complexity and key motif requirements for preserving predictions across different molecules.

3 Conclusion

In this lab, you explored how Graph Neural Networks learn from molecular structures and how GNNExplainer reveals the specific components of a molecule that drive a model's prediction. After constructing and training a GCN on the MUTAG dataset, you used GNNExplainer to generate local explanations in the form of edge masks, feature masks, and minimal faithful subgraphs. These explanations highlighted chemically meaningful motifs such as NO groups, NH groups, and aromatic ring systems—structural patterns known to correlate with mutagenicity in nitroaromatic compounds.

By visualizing full explanations, top-k important edges, and minimal subgraphs, you gained an understanding of how a GNN makes decisions and how to evaluate the faithfulness of those explanations. The fidelity curve and dataset-wide statistics provided a quantitative view of how much information the model truly relies on when making predictions.

Overall, this lab demonstrated the value of combining graph deep learning with explainability techniques. GNNExplainer not only increases transparency and trust in GNN models but also helps verify that predictions align with established chemical knowledge. The same concepts extend to many domains where interpretability and structural reasoning are essential, including drug discovery, material science, and bioinformatics.

4 References

1. Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *International Conference on Learning Representations (ICLR)*.
2. Ying, Z., Bourgeois, D., You, J., Zitnik, M., & Leskovec, J. (2019). GNNExplainer: Generating Explanations for Graph Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*.
3. The molecular-graph dataset used in this lab is based on the MUTAG nitroaromatic compound benchmark from the TUDataset collection.

End of Report

Confidential - Maibiya R. KhumanCha
Version 1.0 — Generated: March 3, 2025